

CS 650 – Full Stack Application Develop

4-1 Activity: Securing API Routes and Passwords

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

July 28, 2026

Module Four Activity: Securing API Routes and Passwords

Authentication and authorization are essential security controls that protect sensitive information throughout the request and response life cycle. Authentication verifies the identity of a user before allowing access to protected resources, while authorization determines what actions an authenticated user is permitted to perform. Together, these controls ensure that only authorized users can access confidential application data.

Authentication begins when a user submits a username and password through the login page. The application compares the submitted password with the securely stored password hash. If the credentials are valid, the server creates a session containing the authenticated user's identity and assigned role. On every protected request, authentication of middleware verifies that a valid session exists before allowing the request to continue. If no valid session is found, the server immediately returns a 401 Unauthorized response. This protects sensitive information by preventing anonymous users from accessing client records, order information, or other protected resources. Authorization takes place after authentication has been completed. Once the application confirms the user's identity, it checks the user's assigned role to determine whether the requested action is permitted. In this application, salespersons can view only their own client lists and submit orders, while managers can view all client records, review all orders, and approve or reject pending orders. If an authenticated salesperson attempts to perform a manager-only action, the server returns a 403 Forbidden response. This prevents privilege escalation and ensures users cannot perform actions outside of their assigned responsibilities.

Sensitive information must also be protected while it is transmitted between the browser and the server. HTTPS uses Transport Layer Security (TLS) to encrypt login credentials, session cookies, customer information, and API responses while they travel across the network. Encryption reduces the risk of attackers intercepting or modifying sensitive data during transmission. Authentication and authorization determine who may access the information, while HTTPS protects that information during transit. Secure session management provides an additional layer of protection. After a successful login, the server creates a new authenticated session and stores only the information necessary for future authorization checks, such as the user's ID and role. Session cookies should use the `HttpOnly` attribute to prevent client-side JavaScript from accessing them and the `Secure` attribute to ensure they are transmitted only over HTTPS connections. These settings help reduce the risk of session hijacking and other session-based attacks.

Password Hashing

Password hashing protects user credentials by ensuring that plaintext passwords are never stored in the application database. During the account creation or database seeding process, each password is combined with a unique random salt and processed using the bcrypt hashing algorithm. Only the generated hash and salt are stored in the database. During login, the submitted password is hashed again using the stored salt. The newly generated hash is then compared with the stored password hash using a timing-safe comparison function. If the values match, authentication succeeds. If they do not match, the login request is rejected. Because passwords are stored only as hashes, an attacker who gains access to the database cannot directly view users' original passwords.

Authentication Controls for Protected API Routes

Authentication controls were implemented by requiring every protected API route to verify that the user has successfully logged in. After authentication, the user's information is stored in the session and reused during future requests. If a request does not include a valid authenticated session, the authentication middleware immediately stops the request and returns a 401 Unauthorized response. This prevents unauthenticated users from accessing protected client information or performing sensitive operations.

Authorization Controls for Protected Actions

Role-based authorization was implemented to control access to sensitive application features. The application defines two user roles:

- **Salesperson**
 - View personal client list
 - Submit orders
- **Manager**
 - View all clients
 - Review all orders
 - Approve pending orders
 - Reject pending orders

Authorization of middleware checks the authenticated user's role before allowing access to manager-only routes. If a salesperson attempts to approve or reject an order, the request is denied with a 403 Forbidden response. Managers, however, are permitted to complete these actions successfully. This layered approach ensures that authentication verifies the user's identity while authorization verifies the user's permissions.

Validation of Security Changes

The completed application was tested to verify that the implemented security controls function correctly.

Password Hashing

The database stores hashed passwords instead of plaintext passwords. The password values are long hashed strings generated using the bcrypt hashing algorithm combined with a unique random salt. This demonstrates that user passwords are securely protected before being stored.

Figure 1: Screenshot showing hashed passwords stored in the database.

```
mysql> use mydb;
mysql> select * from users;
+----+-----+-----+
| id | username | password_hash |
+----+-----+-----+
| 1  | 'alice'  | '$2b$12$4Vt...$...' |
| 2  | 'bob'    | '$2b$12$4Vt...$...' |
+----+-----+-----+
```

Protected Route Rejects Unauthenticated Requests

An API request was made to a protected route without first logging into the application. Because no authenticated session was present, the authentication middleware rejected the request and returned an HTTP 401 Unauthorized response.

This confirms that protected resources cannot be accessed by unauthenticated users.

Figure 2: Screenshot showing a protected route returning 401 Unauthorized.

```
[ ] 1 | bob | 1280233a4d32110a81f07c9a8a810a48776d3758090a8a8080a1c018f4a4a32a88077746c0a8c7c280a809a8087a48f4a8132a78a4a17cfc090a82a4a137a67a11732a8a |
malgorzata@bbsk@Malgorzatas-iMac Module Four Activity Securing API Routes and Passwords % curl -i http://localhost:3000/api/orders
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 29
ETag: W/"1d-T5Z8B2GArgh3Wac3F9MMp"
Date: Tue, 28 Jul 2026 00:02:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5
malgorzata@bbsk@Malgorzatas-iMac Module Four Activity Securing API Routes and Passwords %
```

Protected Route Rejects Unauthorized Requests

A salesperson's account successfully logged into the application and then attempted to access a manager-only route used to approve orders. Although the user was authenticated, the authorization middleware determined that the salesperson's role did not have sufficient permissions and returned an HTTP 403 Forbidden response.

This demonstrates that role-based authorization correctly prevents users from performing actions outside of their assigned responsibilities.

Figure 3: Screenshot showing a salesperson receiving 403 Forbidden when attempting a manager-only action.

```
{ "id": 2, "username": "bob", "role": "salesperson", "full_name": "Bob Smith" }
malgorzata@bbsk@Malgorzatas-iMac Module Four Activity Securing API Routes and Passwords % curl -i -b bob-cookies.txt \
-X PATCH http://localhost:3000/api/orders/2/approve
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 21
ETag: W/"15-TLNzmZqKxaTuFdK/dVWMPBu44/c"
Date: Mon, 27 Jul 2026 23:07:06 GMT
Connection: keep-alive
```

Authorized Manager Request

A manager account successfully authenticated and accessed the same protected approval route. Because the authenticated user had the required manager's role, the application processed the request successfully and returned an HTTP 200 OK response. This confirms that authenticated users with the correct role can access protected functionality while unauthorized users are denied.

Figure 4: Screenshot showing a manager successfully approving an order.

```
malgorzatadebska@Malgorzatas-iMac Module Four Activity Securing API Routes and Passwords % curl -i -c alice-cookies.txt \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"password123"}' \
http://localhost:3000/api/login
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 72
ETag: W/"48-GTXQ5PX/CG64Yjc+VvX078Vbz/4"
Set-Cookie: connect.sid=s%3AvARq5lmo-WvcLqqutjJI6UEWTbfJxcv.ASSfMhBv5ZEJ3bYgngXi%2FL7aSVf6n36g%2FcCSjlUFeoI; Path=/; Expires=Tue, 28 Jul 2026 01:04:54 GMT; HttpOnly; SameSite=Lax
Date: Tue, 28 Jul 2026 00:04:54 GMT
Connection: keep-alive
Keep-Alive: timeout=5

malgorzatadebska@Malgorzatas-iMac Module Four Activity Securing API Routes and Passwords %
```

Additional screenshots:

SalesFlow

Sign in to your account

Username

Password

Sign In

My Clients

NAME	EMAIL	PHONE
Stark Industries	orders@stark.com	555-0101
Oscorp	info@oscorp.com	555-0102
Rand Enterprises	sales@rand.com	555-0103

My Orders

CLIENT	DESCRIPTION	AMOUNT	STATUS
Stark Industries	Arc reactor components	\$25,000.00	Approved
Stark Industries	Vibranium shipment	\$150,000.00	Pending
Oscorp	Goblin glider maintenance	\$8,400.00	Pending

New Order

Client

Select a client...

Amount (\$)

Description

Submit Order

All Clients

NAME	EMAIL	PHONE	SALESPERSON
Stark Industries	orders@stark.com	555-0101	Bob Smith
Oscorp	info@oscorp.com	555-0102	Bob Smith
Rand Enterprises	sales@rand.com	555-0103	Bob Smith
Pym Technologies	hello@pymtech.com	555-0201	Charlie Davis
Damage Control	cleanup@damagectrl.com	555-0202	Charlie Davis

All Orders

CLIENT	DESCRIPTION	AMOUNT	STATUS	SALESPERSON	ACTIONS
Stark Industries	Arc reactor components	\$25,000.00	Approved	Bob Smith	
Stark Industries	Vibranium shipment	\$150,000.00	Pending	Bob Smith	ApproveReject
Oscorp	Goblin glider maintenance	\$8,400.00	Pending	Bob Smith	ApproveReject
Pym Technologies	Pym particle canisters	\$12,000.00	Rejected	Charlie Davis	
Pym Technologies	Ant farm habitat enclosure	\$3,200.00	Pending	Charlie Davis	ApproveReject
Damage Control	Alien debris cleanup crew	\$22,000.00	Approved	Charlie Davis	